



❖ Basic Image Processing using PIL (Python Imaging Library)

By: Mubasshir Ahmed | FSDS Project

❖ Project Overview

This notebook demonstrates **fundamental image processing operations** using the `PIL (Pillow)` library in Python. These transformations form the core of image preprocessing tasks commonly used in:

- ❖ Machine Learning & Deep Learning
- ❖ Computer Vision Applications
- ❖ Photo Editing and Enhancement Pipelines

We will perform the following operations on a sample image:

- ❖ Load and convert image to NumPy array
- ❖ Visualize individual RGB channels
- ❖ Apply basic transformations like rotation and flipping
- ❖ Apply artistic filters such as blur, sharpen, and contour
- ❖ Save and display processed results

❖ Input Image: `3.jpg`

❖ Path: `../input/3.jpg`

❖ Section 2 – Load Image and Convert to NumPy Array

In this section, we'll load an image using the `PIL.Image.open()` method and convert it to a `NumPy` array using `np.asarray()`.

This allows us to directly inspect and manipulate pixel-level data.

- ❖ Understand image shape and channel structure
- ❖ Prepare image for transformations and filtering
- ❖ Visualize using `matplotlib.pyplot`

```
In [2]: # ⬢ Import required libraries
        from PIL import Image
        import numpy as np
        import matplotlib.pyplot as plt

        # ⬢ Load image from input folder
        img_path = '../input/3.jpg'
        img = Image.open(img_path)

        # > Show original image
        plt.imshow(img)
        plt.axis('off')
        plt.title("Original Image")
        plt.show()
```

Original Image



```
In [ ]: # ⬢ Convert PIL image to NumPy array for pixel manipulation
        car_arr = np.asarray(img)

        # > Print shape to understand dimensions (H x W x Channels)
        print("⬢ NumPy array shape:", car_arr.shape)

        # > Optional: Preview the raw array (commented out if too large)
        # print(car_arr)
```

```
[[[219 216 211]
   [219 216 211]
   [219 216 211]
   ...
   [232 222 213]
   [232 222 213]
   [232 222 213]]]
```

```
[[[218 215 210]
   [218 215 210]
   [218 215 210]
   ...
   [232 222 213]
   [232 222 213]
   [232 222 213]]]
```

```
[[[218 215 210]
   [218 215 210]
   [218 215 210]
   ...
   [232 222 213]
   [232 222 213]
   [232 222 213]]]
```

...

```
[[ 64  56  54]
 [ 51  43  41]
 [ 50  42  39]
 ...
 [225 217 204]
 [227 219 208]
 [229 221 210]]]
```

```
[[ 94  86  84]
 [ 69  61  58]
 [ 57  49  46]
 ...
 [224 216 203]
 [229 221 210]
 [233 225 214]]]
```

```
[[125 117 115]
 [ 91  83  80]
 [ 68  60  57]
 ...
 [225 217 204]
 [233 225 214]
 [238 230 219]]]
```

◇ NumPy array shape: (2160, 3840, 3)

❖ Section 3 – Visualize RGB Channels Using NumPy

Each image is composed of 3 color channels:

- ❖ Red
- ❖ Green
- ❖ Blue

Using `NumPy`, we can extract these channels and visualize them individually. This is a great way to understand the structure of image data at a pixel level.

- ❖ Slice 3D image array into separate channels
- ❖ Use different colormaps for clarity
- ❖ Display using `matplotlib`

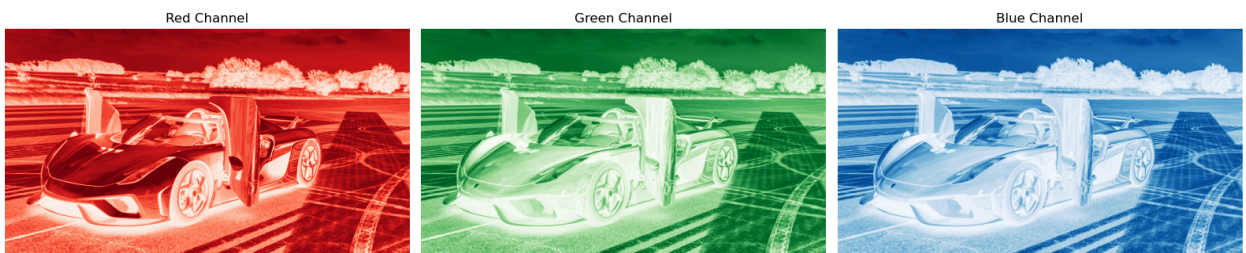
```
In [7]: # ❖ Show RGB channels separately using array slicing
plt.figure(figsize=(16, 5))

# ❖ Red Channel
plt.subplot(1, 3, 1)
plt.imshow(car_arr[:, :, 0], cmap='Reds')
plt.title("Red Channel")
plt.axis('off')

# ❖ Green Channel
plt.subplot(1, 3, 2)
plt.imshow(car_arr[:, :, 1], cmap='Greens')
plt.title("Green Channel")
plt.axis('off')

# ❖ Blue Channel
plt.subplot(1, 3, 3)
plt.imshow(car_arr[:, :, 2], cmap='Blues')
plt.title("Blue Channel")
plt.axis('off')

plt.tight_layout()
plt.show()
```



❖ Section 4 – Rotate and Flip the Image Using PIL

Image transformations such as rotation and flipping are often used for:

- ❖ Data augmentation in Machine Learning
- ❖ Creative manipulations in photo editing
- ❖ Changing the orientation for visual balance

We will apply the following:

- ❖ 90° Rotation using `Image.rotate()`
 - ❖ Vertical Flip using `Image.transpose()`
-

```
In [15]: # ❖ Rotate the image by 90 degrees
rotated_img = img.rotate(90)
rotated_img.save("../output/rotate_90.jpg")

# ❖ Flip the image vertically
flipped_img = img.transpose(Image.FLIP_LEFT_RIGHT)
flipped_img.save("../output/flip_top_bottom.jpg")
```

```
In [16]: # > Display both transformed images
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(rotated_img)
plt.title("Rotated 90°")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(flipped_img)
plt.title("Flipped Vertically")
plt.axis('off')

plt.tight_layout()
plt.show()
```



◇ Section 5 – Apply Filters: Blur, Sharpen, Contour, and More

The `PIL.ImageFilter` module provides built-in filters to enhance or transform images. These are commonly used in:

- ◇ Creative tools and editors
- ◇ Preprocessing for edge/feature detection
- ◇ Improving visual clarity

We will apply the following filters:

- ◇ Blur
- ◇ Sharpen
- ◇ Detail
- ◇ Find Edges
- ◇ Contour

Each filter will be saved and displayed for comparison.

```
In [13]: from PIL import ImageFilter

# ◇ Dictionary of filters to apply
filters = {
    "blur": ImageFilter.BLUR,
    "sharpen": ImageFilter.SHARPEN,
    "detail": ImageFilter.DETAIL,
    "edges": ImageFilter.FIND_EDGES,
    "contour": ImageFilter.CONTOUR
}

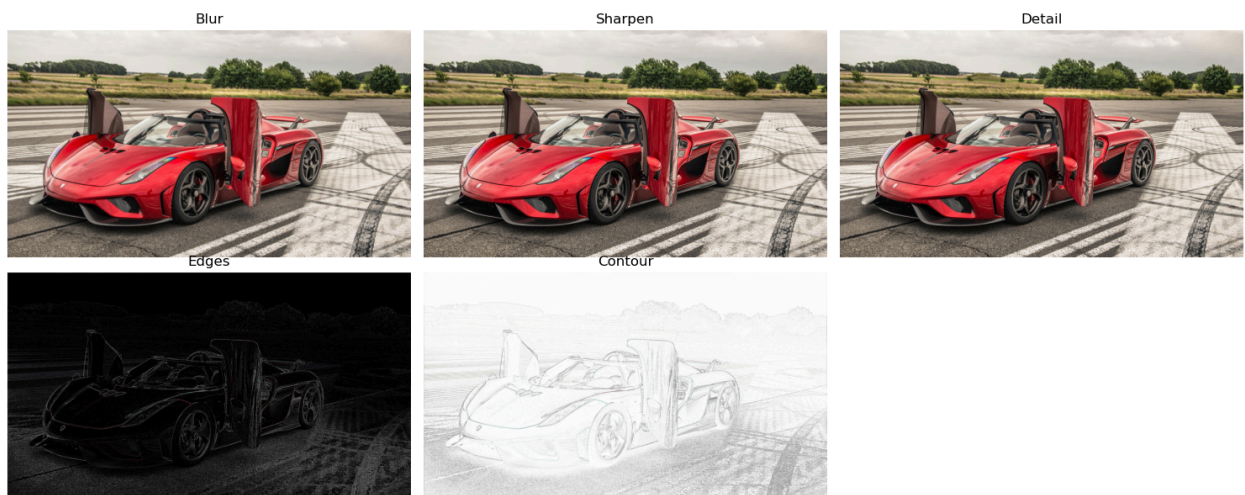
# > Apply, save, and store filtered images
filtered_images = {}
```

```
for name, filter_type in filters.items():
    filtered = img.filter(filter_type)
    filtered.save(f"../output/{name}.jpg")
    filtered_images[name] = filtered
```

```
In [14]: # Display all filters in a grid
plt.figure(figsize=(15, 6))

for i, (name, image) in enumerate(filtered_images.items(), start=1):
    plt.subplot(2, 3, i)
    plt.imshow(image)
    plt.title(name.capitalize())
    plt.axis('off')

plt.tight_layout()
plt.show()
```



❖ Section 6 – Save Results & Wrap-Up

All transformed and filtered images were saved to the `output/` folder with meaningful filenames. These saved files can be used for:

- ❖ Further machine learning training/testing
- ❖ Creative edits
- ❖ Visual documentation of processing pipelines

❖ Summary:

In this notebook, we explored the basic yet powerful image processing capabilities of the `PIL (Pillow)` library using Python:

- ◇ Loaded images using `Image.open()`
 - ◇ Visualized RGB channels using NumPy
 - ◇ Rotated and flipped images
 - ◇ Applied artistic filters such as blur, sharpen, contour
 - ◇ Saved all output images to a clean folder
-

◇ What's Next?

- Try combining multiple filters and transformations
- Chain PIL with OpenCV for more advanced tasks
- Start learning about color enhancement and AI-based stylization

Project by: Mubasshir Ahmed **Guided by:** K. Prakash sir - FSDS
Project
